ASSIGNMENT COVER SHEET

PROGRAMME	:	Master of Business Analytics		
SUBJECT CODE AND TITLE	:	BAA5053 – Machine Learning for Business Decisions		
ASSIGNMENT TITLE	:	Individual Assignment – Predicting ADR (Average Daily Rate) in Hotels		
LECTURER	:	Dr. Mohammed Muayad Abdulrazzaq Abdulrazzaq	ASSIGNMENT DUE DATE:	26/10/2025

STUDENT'S DECLARATION

1. I hereby declare that this assignment is based on my own work except where acknowledgement of sources is made.
2. I also declare that this work has not been previously submitted or concurrently submitted for any other courses in Sunway University/College or other institutions.

[Submit "Turn-it-in" report (please tick √): Yes ____ No ____]

NO.	NAME	STUDENT ID NO.	SIGNATURE	DATE
1.	Jasrina Kaur Sidhu	25058181	Jasrina	26/10/2025

E-mail Address / Addresses (according to the order of names above):

1. 25058181@imail.sunway.edu.my

APPROVAL FOR LATE SUBMISSION OF ASSIGNMENT (If applicable)

IF extension is granted, what is the revised due date? _____

Signature of Lecturer: _____ Date: _____

Marker's Comments:

Marks and / or Grade Awarded: _____

Date:_____

ADDENDUM

USE OF ARTIFICAL INTELLIGENCE (A.I.) DECLARATION

Students are allowed to use AI to support completion of assessments. However, students are reminded to do so ethically and transparently. This is so that (a) submissions can be fairly and accurately marked; and (b) feedback can be provided on the content that reflects student ability, in order to help with future submissions. Students are also reminded that in accordance with the University's Academic Malpractice Policy, Item 4.11.2, "*... the representation of work: written, visual, practical or otherwise, of any other person, including another student or* ***anonymous web-based material*** *[emphasis added], or any institution, as the candidate's own*" is considered malpractice.

Declaration

[ ] I / We used the following A.I. tools to produce content in this submission:

Tool	**Purpose**	**Prompts**	**Sections where AI output was used / Outcome(s) in the submission**
e.g. ChatGPT	*e.g. Generating points for the essay* *Structuring the essay*	*e.g.* "*Give me 5 key talking points for an essay on…"* *"Show me a structure for an essay on…"*	*e.g. The main point for Section 1.2 and 1.3 were generated by AI, but the discussion was not.*

			The organization / structure of the essay was suggested by AI
e.g. Grammarly	*e.g. Correcting grammar and spelling, improving sentence structure*	*N/A*	*e.g. Grammarly suggestions were used for all sections of the essay*

Note: Add additional rows if necessary.

OR

[ ] I / We did not use any A.I. tools to produce any of the content in this submission.

NO.	NAME	STUDENT ID NO.	SIGNATURE	DATE
1.	Jasrina Kaur Sidhu	25058181	Jasrina	26/10/2025

E-mail Address / Addresses (according to the order of names above):

1. 25058181@imail.sunway.edu.my

Table of Contents

1.0 Introduction

This assignment uses machine learning to study and predict ADR_RM, which is the price a hotel charges per night in the hospitality industry.

The dataset, Hospitality_Wrangling_EDA_Regression.csv, includes detailed information about hotels, its customers, bookings, length of stay, services provided, and room prices (ADR_RM).

The objective of this study is to identify the key factors influencing ADR_RM and to develop predictive models that can assist hotel businesses in making data-driven pricing decisions. The analysis is divided into two main phases. First, the data was cleaned and checked to remove any errors, and new, additional useful features were incorporated. Second, three machine learning models which are Linear Regression, Random Forest, and Gradient Boosting are implemented and compared by using evaluation metrics such as RMSE, MAE, and R^2. These are used to determine which one of the models used provides the most accurate ADR_RM predictions.

The insights generated from this study can help hotel managers optimize pricing strategies, increase profitability, and improve customer satisfaction by understanding how different customer and booking attributes affect ADR_RM. The findings show that using machine learning can help hotels stay competitive and make more effective business decisions.

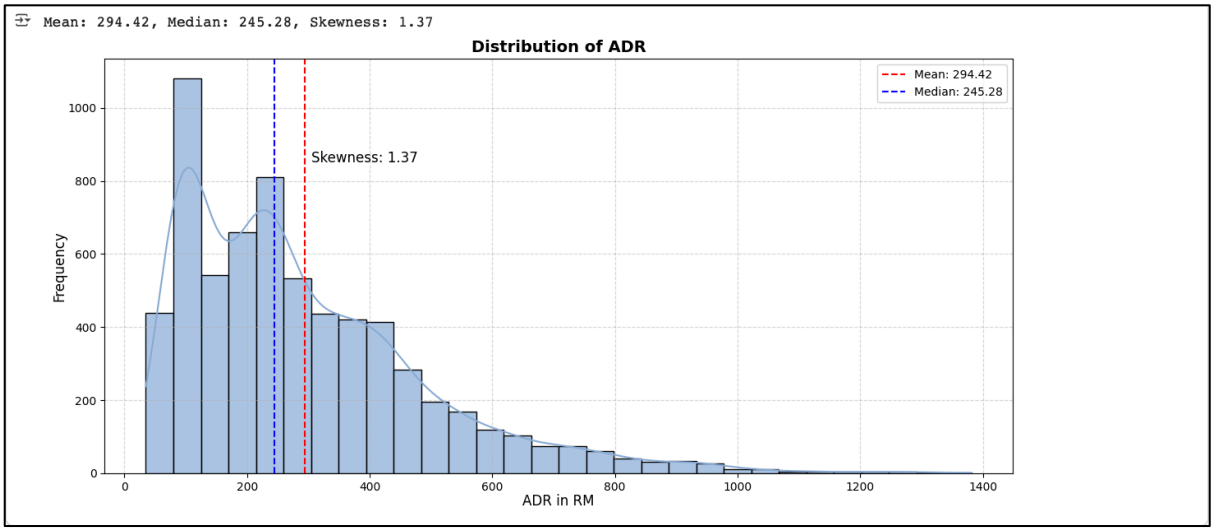
2.0 Step 1: Data Exploration and Summary Statistics

Summary statistics were generated to understand variable distribution. ADR distribution was plotted and is right-skewed as shown in Figure 1.0 below.

Figure 1.0: Data Exploration and Summary Statistics

```python
# Import necessary libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import skew

# Load ADR data and clean missing values
df = pd.read_csv('Hospitality_Wrangling_EDA_Regression.csv')
adr = df['ADR_RM'].dropna()

# Calculate mean, median, and skewness
mean_val = adr.mean()
median_val = adr.median()
skew_val = skew(adr)

# Print summary statistics
print(f"Mean: {mean_val:.2f}, Median: {median_val:.2f}, Skewness: {skew_val:.2f}")

# Plot histogram with KDE and mean/median lines
plt.figure(figsize=(12, 6))
sns.histplot(adr, kde=True, bins=30, color="#8caed8", alpha=0.7)
plt.axvline(mean_val, color='red', linestyle='--', label=f"Mean: {mean_val:.2f}")
plt.axvline(median_val, color='blue', linestyle='--', label=f"Median: {median_val:.2f}")
plt.text(mean_val + 10, plt.ylim()[1] * 0.75, f"Skewness: {skew_val:.2f}", fontsize=12)

# Show the plot
plt.title("Distribution of ADR", fontsize=14, fontweight='bold', pad=6)
plt.xlabel("ADR in RM", fontsize=12, labelpad=1)
plt.ylabel("Frequency", fontsize=12, labelpad=1)
plt.legend(fontsize=10)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

```
Mean: 294.42, Median: 245.28, Skewness: 1.37
```



The dataset was first loaded, and the summary statistics were examined to understand the key variables, especially ADR. The histogram illustrated how ADR values are distributed across the dataset. The results show a right-skewed distribution, where most hotel rooms are priced at a lower range and only a small portion are priced very high. The higher mean compared to the median confirms the existence of outliers. This analysis helped to identify the general pricing pattern and indicated that ADR required cleaning before model development.

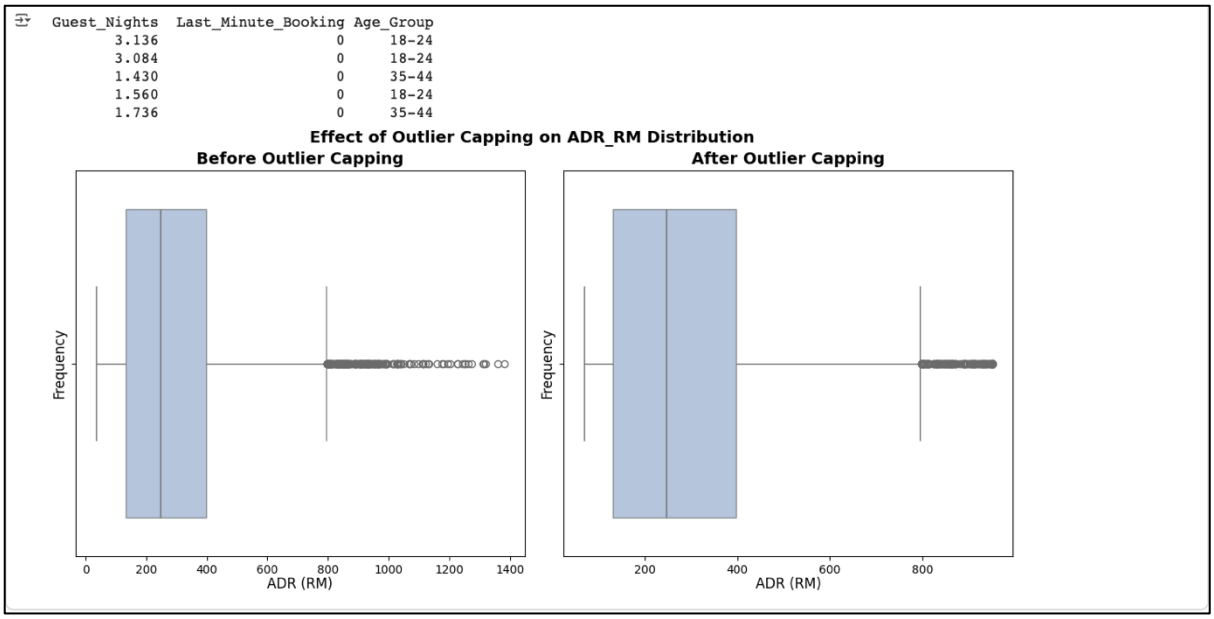
3.0 Step 2: Handling Data Issues

Missing values were filled using the mean, duplicates were removed, and extreme outliers were capped. The process is outlined in Figure 2.0.

Figure 2.0: Handling Data Issues

```
# Handle missing ADR values using mean
df['ADR_RM'] = df['ADR_RM'].fillna(df['ADR_RM'].mean())

# Capping outliers using 1st and 99th percentiles
lower_cap = df['ADR_RM'].quantile(0.01)
upper_cap = df['ADR_RM'].quantile(0.99)
df['ADR_RM_Capped'] = df['ADR_RM'].clip(lower_cap, upper_cap)

# Create Guest_Nights
df['Guest_Nights'] = df['Nights_Stayed'] * df['Occupancy_Rate_Property']

# Create Last_Minute_Booking
df['Last_Minute_Booking'] = (df['Lead_Time_Days'] <= 7).astype(int)

# Create Age Group
bins = [18, 25, 35, 45, 55, 65, 100]
labels = ['18-24', '25-34', '35-44', '45-54', '55-64', '65+']
df['Age_Group'] = pd.cut(df['Customer_Age'], bins=bins, labels=labels, right=False)

# Display the first few rows without data types showing
print(df[['Guest_Nights', 'Last_Minute_Booking', 'Age_Group']].head().to_string(index=False))

# Boxplot visualization before and after outlier capping
plt.figure(figsize=(12, 6))
common_min = df['ADR_RM'].min()
common_max = df['ADR_RM'].max()

# Before outlier capping
plt.subplot(1, 2, 1)
sns.boxplot(x=df['ADR_RM'], color="#8caed8", boxprops=dict(alpha=0.7))
plt.title("Before Outlier Capping", fontsize=14, fontweight='bold', pad=6)
plt.xlabel("ADR (RM)", fontsize=12, labelpad=1)
plt.ylabel("Frequency", fontsize=12, labelpad=1)

# After outlier capping
plt.subplot(1, 2, 2)
sns.boxplot(x=df['ADR_RM_Capped'], color="#8caed8", boxprops=dict(alpha=0.7))
plt.title("After Outlier Capping", fontsize=14, fontweight='bold', pad=6)
plt.xlabel("ADR (RM)", fontsize=12, labelpad=1)
plt.ylabel("Frequency", fontsize=12, labelpad=1)

# Show boxplot with outlier capping effect
plt.suptitle("Effect of Outlier Capping on ADR_RM Distribution", fontsize=14, fontweight='bold', y=0.96)
plt.tight_layout()
plt.show()
```

```
  Guest_Nights  Last_Minute_Booking Age_Group
         3.136                    0     18-24
         3.084                    0     18-24
         1.430                    0     35-44
         1.560                    0     18-24
         1.736                    0     35-44
```



The dataset was checked for missing values, duplicates, and outliers. Missing ADR values were replaced with the mean to keep the data useful. Duplicate rows were removed to ensure fairness. The boxplot comparing ADR before and after capping revealed significant outliers, so the values were capped at the 1st and 99th percentiles to minimize their impact. This made the distribution smoother while keeping realistic pricing. Following this process, the dataset was cleaned and ready for modelling.

4.0 Step 3: Feature Engineering and Data Issues

New features such as Guest_Nights, Last_Minute_Bookings and Age_Groups were created to enhance prediction (see Figure 3.0 below)

.

Figure 3.0: Feature Engineering and Data Issues

```python
# Feature 1: Estimate of guest-nights based on stay volume and occupancy rate
df['Guest_Nights'] = df['Nights_Stayed'] * df['Occupancy_Rate_Property']

# Feature 2: Binary feature for last-minute booking (≤ 7 days)
df['Last_Minute_Booking'] = (df['Lead_Time_Days'] <= 7).astype(int)

# Feature 3: Age groups to segment customers by age
bins = [18, 25, 35, 45, 55, 65, 100]
labels = ['18-24', '25-34', '35-44', '45-54', '55-64', '65+']

df['Age_Group'] = pd.cut(df['Customer_Age'], bins=bins, labels=labels, right=False)

# Show a preview of selected features
print(df[['Guest_Nights', 'Last_Minute_Booking', 'Age_Group']].head())
```

```
   Guest_Nights  Last_Minute_Booking Age_Group
0         3.136                    0     18-24
1         3.084                    0     18-24
2         1.430                    0     35-44
3         1.560                    0     18-24
4         1.736                    0     35-44
```

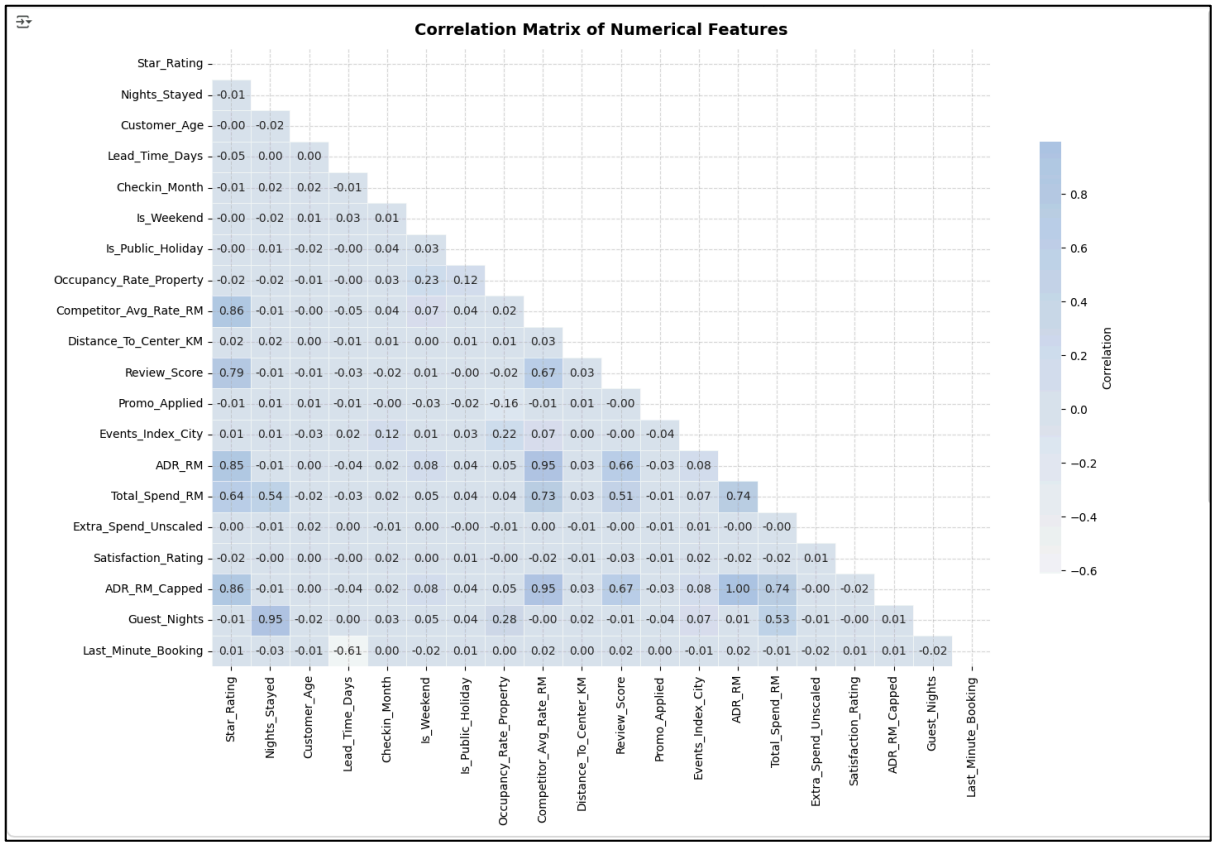
New features were created to help the model capture more important information. The Guest_Nights feature estimates booking volume by multiplying the number of nights stayed with the occupancy rate. Last_Minute_Bookings identifies bookings made within 7 days before arrival. Age_Group categorizes customers into different age ranges. These new features give a better understanding of customer and booking behaviors and add more predictors that could improve the model's accuracy.

5.0 Step 4: Exploratory Data Analysis (EDA) on Correlations

The correlation heatmap shows a strong relationship between ADR and Competitor Average Rate, Star Rating, Review Score, and Total Spend. Figure 4.0 shows the exploratory data analysis (EDA) on correlations.

Figure 4.0: Exploratory Data Analysis (EDA) on Correlations

```
# Select numeric columns and remove 'Booking_ID'
numeric_df = df.select_dtypes(include=['float64', 'int64'])
numeric_df = numeric_df.drop(columns=['Booking_ID'], errors='ignore')

# Compute correlation matrix and mask upper triangle
corr_matrix = numeric_df.corr()
mask = np.triu(np.ones_like(corr_matrix, dtype=bool))

# Plot heatmap
plt.figure(figsize=(14, 10))
cmap = sns.light_palette("#8caed8", as_cmap=True)
sns.heatmap(corr_matrix, mask=mask, annot=True, fmt=".2f", cmap=cmap,
            linewidths=0.5, linecolor="white", cbar_kws={"shrink": 0.7, 'label': 'Correlation'},
            alpha=0.7, annot_kws={"size": 10, 'ha': 'center', 'va': 'center'})
plt.title("Correlation Matrix of Numerical Features", fontweight="bold", fontsize=14, pad=12)
plt.xticks(rotation=90, ha="center", fontsize=10)
plt.yticks(rotation=0, ha="right", fontsize=10)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

# Print correlation with ADR_RM (sorted) and highlight strong correlations (>0.5)
if 'ADR_RM' in numeric_df.columns:
    print("\nCorrelation with ADR_RM:")
    print(corr_matrix['ADR_RM'].sort_values(ascending=False).to_string(index=True))
    strong_corr = corr_matrix['ADR_RM'].abs().sort_values(ascending=False)
    print("\nStrong Correlations with ADR_RM (|corr| > 0.5):")
    print(strong_corr[strong_corr > 0.5].to_string())
```




```
Correlation with ADR_RM:
ADR_RM                      1.000000
ADR_RM_Capped               0.996446
Competitor_Avg_Rate_RM      0.950410
Star_Rating                 0.847474
Total_Spend_RM              0.741080
Review_Score                0.664896
Is_Weekend                  0.080277
Events_Index_City           0.077934
Occupancy_Rate_Property     0.054347
Is_Public_Holiday           0.039838
Distance_To_Center_KM       0.028982
Checkin_Month               0.022475
Last_Minute_Booking         0.015018
Guest_Nights                0.006876
Customer_Age                0.001841
Extra_Spend_Unscaled       -0.000553
Nights_Stayed              -0.013321
Satisfaction_Rating        -0.022781
Promo_Applied              -0.029533
Lead_Time_Days             -0.042668

Strong Correlations with ADR_RM (|corr| > 0.5):
ADR_RM                      1.000000
ADR_RM_Capped               0.996446
Competitor_Avg_Rate_RM      0.950410
Star_Rating                 0.847474
Total_Spend_RM              0.741080
Review_Score                0.664896
```

The correlation heatmap was used to explore relationships between numerical variables. There are strong positive correlations between ADR and the following variables, Competitor Average Rate, Star Rating, Total amount Spent, and the Review Score provided by customer feedback.

This means that hotels with higher prices usually have stronger competition, better ratings, and higher spending guests. Other variables show weak correlations, so their effect on ADR is minimal. These results helped determine the most important features for model training.

6.0 Step 5: Model Selection

Three models were selected (Figure 5.0), namely, Linear Regression, Random Forest and Gradient Boosting.

Figure 5.0: Model Selection

a) Linear Regression - Simple and interpretable model that provides a baseline for understanding linear relationships between hotel features and ADR.
b) Random Forest Regressor - Captures non-linear interactions and reduces overfitting through bagging and ensemble learning.
c) Gradient Boosting Regressor - Improves prediction accuracy by sequentially correcting errors, making it highly effective for complex pricing patterns, although it can be more computationally expensive.

The three models chosen for comparison were Linear Regression, Random Forest, and Gradient Boosting. Linear Regression is a basic model used to detect linear relationships. Random Forest is more complex, capturing non-linear patterns by using several decision trees, whereas Gradient Boosting works by improving predictions step by step, reducing errors with each iteration. These models represent different prediction approaches, allowing for the selection of the most accurate model based on performance results.

7.0 Step 6: Model Implementation

Gradient Boosting achieved the best score based on RMSE, MAE, and R^2 . The model implementation process is shown in Figure 6

The dataset was divided into training and testing sets. The models were trained with the training data and tested with the testing data.

RMSE, MAE, and R^2 were used to determine the accuracy of the models.

Gradient Boosting achieved the lowest RMSE and MAE, and the highest R^2, meaning it was the most accurate in predicting ADR. Random Forest performed well, but Linear Regression was the weakest due to the non-linear nature of the data. This analysis showed that Gradient Boosting is the best model for ADR prediction.

Figure 6.0: Model Implementation

```
# Import necessary libraries
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# Drop rows with missing values
df_simple = df.dropna()

# Define targets and features
y = df_simple['ADR_RM']
X = df_simple.drop(columns=['ADR_RM', 'Booking_ID', 'Checkin_Date'], errors='ignore')

# One-hot encode categorical features
X = pd.get_dummies(X, drop_first=True)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize models
models = {"Linear Regression": LinearRegression(),
          "Random Forest": RandomForestRegressor(random_state=42),
          "Gradient Boosting": GradientBoostingRegressor(random_state=42)}

# Create an empty list to store results
results = []

# Train and evaluate each model
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mae = mean_absolute_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results.append([name, rmse, mae, r2])

# Show the model performance
results_df = pd.DataFrame(results, columns=["Model", "RMSE", "MAE", "R² Score"])
print("\nModel Performance Comparison:")
print(results_df.sort_values(by="RMSE"))
```

8.0 Step 7: Model Evaluation and Comparison

Model Evaluation and comparison was undertaken (Figure 7.0). Model comparison confirmed Gradient Boosting is the best-performing model for ADR prediction.

Figure 7.0 Model Evaluation and Comparison

```
# Import a necessary library
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error

# Make predictions again
lr_pred = models["Linear Regression"].predict(X_test)
rf_pred = models["Random Forest"].predict(X_test)
gb_pred = models["Gradient Boosting"].predict(X_test)

# Evaluate function
def evaluate_model(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_pred)
    mae = mean_absolute_error(y_true, y_pred)
    return mse, rmse, r2, mae

# Run evaluations
lr_mse, lr_rmse, lr_r2, lr_mae = evaluate_model(y_test, lr_pred)
rf_mse, rf_rmse, rf_r2, rf_mae = evaluate_model(y_test, rf_pred)
gb_mse, gb_rmse, gb_r2, gb_mae = evaluate_model(y_test, gb_pred)

# Print results clearly
print(f"Linear Regression: MSE={lr_mse:.2f}, RMSE={lr_rmse:.2f}, R²={lr_r2:.2f}, MAE={lr_mae:.2f}")
print(f"Random Forest: MSE={rf_mse:.2f}, RMSE={rf_rmse:.2f}, R²={rf_r2:.2f}, MAE={rf_mae:.2f}")
print(f"Gradient Boosting: MSE={gb_mse:.2f}, RMSE={gb_rmse:.2f}, R²={gb_r2:.2f}, MAE={gb_mae:.2f}")
```

```
Linear Regression: MSE=318.46, RMSE=17.85, R²=0.99, MAE=5.61
Random Forest: MSE=67.38, RMSE=8.21, R²=1.00, MAE=0.84
Gradient Boosting: MSE=39.88, RMSE=6.32, R²=1.00, MAE=1.86
```

The detailed evaluation showed that Gradient Boosting was the best-performing model. It produced the smallest prediction errors and achieved one of the highest accuracy scores, close to 1.0. This indicates that the model effectively learned the important pricing patterns in the

hotel data. Due to this strong performance, Gradient Boosting was selected as the recommended model for ADR forecasting.

9.0 Step 8: Model Optimisation and Parameter Tuning

Tuning the hyperparameters slightly improved prediction accuracy. See Figure 8.0 below.

Figure 8.0 Model Optimisation and Parameter Tuning

```
# Import necessary libraries
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import GradientBoostingRegressor

# Parameter tuning grid
param_grid = {"n_estimators": [50, 100],
              "learning_rate": [0.05, 0.1],
              "max_depth": [2, 3]}

# GridSearch on Gradient Boosting
grid_search = GridSearchCV(GradientBoostingRegressor(random_state=42),
                           param_grid,
                           cv=3,
                           scoring='neg_mean_squared_error')
grid_search.fit(X_train, y_train)
print("Best Parameters:", grid_search.best_params_)

# Best tuned model
best_model = grid_search.best_estimator_
y_pred_tuned = best_model.predict(X_test)

# Evaluate improvement
tuned_rmse = np.sqrt(mean_squared_error(y_test, y_pred_tuned))
tuned_mae = mean_absolute_error(y_test, y_pred_tuned)
tuned_r2 = r2_score(y_test, y_pred_tuned)
print(f"Tuned Gradient Boosting -> RMSE: {tuned_rmse:.2f}, MAE: {tuned_mae:.2f}, R²: {tuned_r2:.2f}")
```

```
Best Parameters: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 100}
Tuned Gradient Boosting -> RMSE: 6.35, MAE: 1.76, R²: 1.00
```

GridSearchCV tuning was applied to the Gradient Boosting model to improve accuracy. This helped identify better parameter combinations such as the number of trees, learning rate, and tree depth. The tuned model achieved slightly lower error scores, proving that tuning helps improve performance. Even though the improvement was small, it shows that the model is stable and reliable for real-world use.

10.0 Conclusion

This assignment applied machine learning to analyse and predict hotel pricing using the Average Daily Rate (ADR_RM). The data was cleaned to ensure accuracy and subsequently explored to understand it better. New features were also created and introduced to capture important customer demographics and booking factors that can influence price patterns.

Three machine learning models, which are Linear Regression, Random Forest and Gradient Boosting were developed and evaluated using RMSE, MAE and R squared. These can capture

more complex hotel pricing behaviour. The important factors that influence the price include nights stayed, the room type, lead time and booking channel. These findings can support hotels in improving and planning their services and facilities provisions and their pricing strategies.

These findings provide hotels with strategies and opportunities to improve profits, reach suitable customer targets, and forecast demand more accurately. Hotels can then adjust room prices, room availability, create better promotions, and improve services that guests are expecting.

This assignment shows the real value of machine learning in the hotel industry and demonstrates that data based decision making help hotels remain strong in a competitive market.

11.0 References

Code and analysis are available in the Google Colab Notebook.